FULL IMPLEMENTATION BLUEPRINT · VERSION 3.0

AegisQA

AI-Powered Test Automation Orchestrator

An enterprise-grade, AI-native QA orchestration platform that transforms project tickets into validated Robot Framework tests, investigates failures with multi-signal evidence correlation, and continuously improves through persistent memory and knowledge retrieval.

VERSION	STATUS	DATE	AUDIENCE
3.0 — Merged Blueprint	Pre-Implementation	June 2026	PM / Technical Review

Table of Contents

AegisQA v3 — Full Implementation Blueprint

AI-Powered Test Automation Orchestrator
Version: 3.0 | **Status:** Pre-Implementation

Table of Contents

1. Vision

Build an enterprise-grade, AI-native QA orchestration platform that transforms Jira and Azure DevOps tickets into validated Robot Framework automation scripts, executes those scripts across environments, investigates failures using multi-signal evidence correlation, and continuously improves through persistent memory and knowledge retrieval — while keeping a human in the loop for every consequential decision.

The system gets smarter with every test run. Past failures inform future investigations. Past fixes inform future self-healing. The platform is not a static tool — it is a learning QA system.

2. Problem Statement

Modern QA teams spend a significant portion of their time on work that is repetitive, context-dependent, and only partially requires human judgment:

- Parsing tickets and extracting testable requirements from loosely written acceptance criteria
- Writing test cases across functional, negative, boundary, and regression dimensions
- Converting test cases into Robot Framework automation scripts with correct syntax and library usage
- Managing test data: seeding environments, handling credentials, running teardown after execution
- Investigating failures by manually correlating logs, API responses, network traces, and screenshots
- Maintaining scripts when UI locators or API contracts change
- Translating technical failure details into executive-readable reports

Existing automation frameworks (Robot Framework, Selenium, Pytest) are reliable executors but have no understanding of business requirements, no causal reasoning about failures, no memory of what was done before, and no self-repair capability.

AI-assisted testing tools exist but are cloud-only, closed-source, disconnected from the Robot Framework ecosystem, and do not learn from past runs.

AegisQA addresses all of these gaps.

---

3. Competitive Landscape

Tool	Strengths	Gaps Relative to AegisQA
Mabl	AI self-healing, polished UI, cloud-native	Cloud-only, no local model support, no Robot Framework output
Testim	Fast UI test recording, self-healing locators	UI-only, no API testing, no failure investigation
Functionize	NLP-based test creation from plain text	Enterprise pricing, closed-source, no multi-agent architecture
Tricentis Tosca	Model-based testing, enterprise integrations	Different paradigm, expensive licensing, no LLM integration
Katalon Studio	Robot Framework support, affordable	No AI generation, no failure intelligence, no memory

AegisQA differentiates on five axes:

1. **Robot Framework-native** output that integrates into existing QA pipelines without migration or retraining
2. **Self-hostable** with local model support via Ollama — no mandatory cloud dependency or data leaving the organization
3. **Multi-signal failure investigation** combining logs, API responses, database state, and network traces with evidence-based (not LLM-hallucinated) confidence scoring
4. **Persistent memory** — the system accumulates knowledge across runs; past failures improve future investigations, past fixes improve future self-healing
5. **Open architecture** — pluggable tools, skills, and agents; new testing capabilities added without modifying the core

---

4. Design Principles

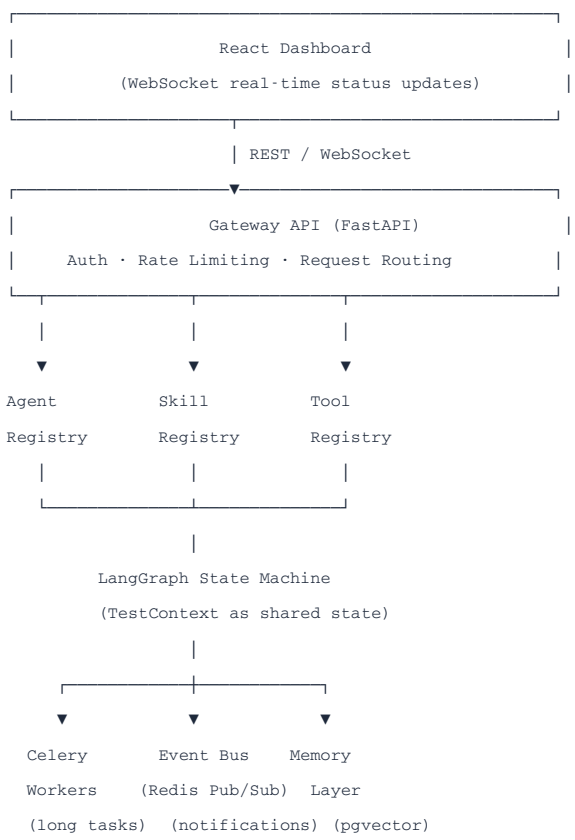
These principles govern every implementation decision. When in doubt, they are the tiebreaker.

- **Deterministic orchestration.** The LangGraph state machine defines which agent runs when. No agent acts outside its defined node. Execution flow is inspectable, replayable, and debuggable.
- **Strict layer boundaries.** Agents invoke Skills. Skills invoke Tools. Agents never call external systems directly. This makes every layer independently testable.
- **Single ownership per TestContext section.** Every agent writes only to its designated section of the shared state. No agent reads uncommitted work from another agent's section.
- **Memory is retrieved, not copied.** Past context is surfaced via semantic search and injected into agent prompts as grounded references — not copied wholesale into state.
- **Human approval before consequences.** No code is committed, no locator is changed, no test is executed in a shared environment without explicit human approval. The system suggests; humans decide.
- **Git-native workflow.** All generated test artifacts live in version control from the moment of approval. The platform never becomes a silo.
- **Explainable AI.** Every confidence score, every root-cause assessment, every locator suggestion must be traceable to specific evidence. No black-box verdicts.
- **Fail loud, not silently.** When an agent fails, retries are attempted (max 2). If still failing, the item is flagged for manual intervention. The system never silently skips a step.
- **Skills are reusable across agents.** A `LogAnalysisSkill` used by the Investigation Agent can be reused by the Self-Healing Agent. Skills are not owned by agents.
- **Tools are stateless and isolated.** A Tool performs one operation and returns a result. It holds no state between calls. It runs in an isolated execution context.

---

5. System Architecture Overview

System Layers



High-Level Workflow

```
    ▼
[Node 06] AutomationGenerator ──────────── Generate .robot + resource files
    │                                      Syntax validation (robot --dryrun)
    │                                      Keyword existence check
    │                                      Store in TestContext.automation
    │
    ├── [FAIL] Retry up to 2x with error injected into prompt
    │          If still failing → flag for manual intervention
    ▼
[Node 07] Validator ──────────────────── Final automated check before human review
    │
    ├── [FAIL] → back to AutomationGenerator
    ▼
[Node 08] HumanApproval ──────────────── Review UI shown to assigned engineer
    │                                      Inline editing, approve/reject per test
    │                                      On approval: Git branch + PR created
    │
    ├── [REJECTED] → back to TestCaseGenerator or AutomationGenerator
    ▼
[Node 09] ExecutionDispatcher ──────────── Dispatch to Celery workers
    │                                      Dashboard trigger or CI/CD webhook
    │                                      Isolated execution per test
    │                                      Collect logs, screenshots, JUnit XML
    ▼
[Node 10] InvestigationCoordinator ──────── Correlate evidence per failed test
    │                                      Evidence scoring (deterministic)
    │                                      Retrieve similar past failures from memory
    ▼
[Node 11] ReportGenerator ──────────────── Technical report for QA
    │                                      Executive summary for stakeholders
    ▼
[Node 12] ArchiveMemory ────────────────── Embed and store:
                                           - TestContext snapshot
                                           - Investigation findings
                                           - Approved scripts
                                           - Update vector index
```

---

6. LangGraph Orchestration Engine

Why LangGraph

LangGraph was chosen over CrewAI and AutoGen for three specific reasons:

- **Explicit state graph.** The execution flow is a directed graph with named nodes and defined edges. Every step is visible and inspectable in the LangGraph Studio debugger.
- **Shared state model.** All nodes operate on a single typed `TestContext` state object. No implicit message passing between agents.
- **Conditional edges.** Branching logic (retry on failure, loop back on rejection, skip self-healing if no failures) is a first-class concept, not a workaround.

Graph Definition

```
from langgraph.graph import StateGraph
```

```
from backend.graph.state import TestContext

workflow = StateGraph(TestContext)

# Add nodes
workflow.add_node("load_ticket",           nodes.load_ticket)
workflow.add_node("requirement_agent",     nodes.requirement_agent)
workflow.add_node("coverage_planner",      nodes.coverage_planner)
workflow.add_node("test_case_generator",   nodes.test_case_generator)
workflow.add_node("test_data_resolver",    nodes.test_data_resolver)
workflow.add_node("automation_generator",  nodes.automation_generator)
workflow.add_node("validator",             nodes.validator)
workflow.add_node("human_approval",        nodes.human_approval)
workflow.add_node("execution_dispatcher",  nodes.execution_dispatcher)
workflow.add_node("investigation_coord",   nodes.investigation_coordinator)
workflow.add_node("report_generator",      nodes.report_generator)
workflow.add_node("archive_memory",        nodes.archive_memory)

# Linear edges
workflow.set_entry_point("load_ticket")
workflow.add_edge("load_ticket",          "requirement_agent")
workflow.add_edge("requirement_agent",    "coverage_planner")
workflow.add_edge("coverage_planner",     "test_case_generator")
workflow.add_edge("test_case_generator",  "test_data_resolver")
workflow.add_edge("test_data_resolver",   "automation_generator")
workflow.add_edge("investigation_coord",  "report_generator")
workflow.add_edge("report_generator",     "archive_memory")

# Conditional edges
workflow.add_conditional_edges(
    "automation_generator",
    routing.route_after_generation,
    {
        "validation_pass":  "validator",
        "retry":            "automation_generator",
        "manual_flag":      END,
    }
)

workflow.add_conditional_edges(
    "validator",
    routing.route_after_validation,
    {
        "pass":         "human_approval",
        "fail":         "automation_generator",
    }
)

workflow.add_conditional_edges(
    "human_approval",
    routing.route_after_approval,
    {
        "approved":         "execution_dispatcher",
        "rejected_regen":   "automation_generator",
        "rejected_retest":  "test_case_generator",
```

```
    }
)

workflow.add_conditional_edges(
    "execution_dispatcher",
    routing.route_after_execution,
    {
        "failures_exist":   "investigation_coord",
        "all_passed":       "report_generator",
    }
)
```

Node Responsibilities Summary

Node	Reads From	Writes To
LoadTicket	Jira/Azure DevOps API	`context.ticket`
RequirementAgent	`context.ticket`, memory, domain docs	`context.requirement_analysis`
CoveragePlanner	`context.requirement_analysis`, past tickets	`context.coverage_plan`
TestCaseGenerator	`context.requirement_analysis`, `context.coverage_plan`	`context.test_cases`
TestDataResolver	`context.test_cases`, Vault, DB	`context.test_data`
AutomationGenerator	`context.test_cases`, `context.test_data`, memory	`context.automation`
Validator	`context.automation`	`context.automation.validation`
HumanApproval	`context.automation` (read)	Git PR URL, approval status
ExecutionDispatcher	`context.automation`, Git	`context.execution`
InvestigationCoordinator	`context.execution`, logs, memory	`context.evidence`
ReportGenerator	`context.evidence`, `context.execution`	`context.reports`
ArchiveMemory	full `context`	Vector DB, episodic memory store

7. OpenClaw-Inspired Runtime

Architecture

The runtime provides the infrastructure that all agents, skills, and tools operate within. Agents never call external systems directly. The call chain is always:

```
Agent → Skill → Tool → External System
```

This ensures every external call is logged, auditable, rate-limited, and can be mocked in tests.

Components

GATEWAY API

The single entry point for all external requests. Handles:

- JWT authentication and session management
- Role-based access control (Admin, QA Engineer, Viewer)
- Request routing to appropriate handlers
- Rate limiting per user and per organization
- Audit logging of all inbound requests

AGENT REGISTRY

A runtime registry of all available agents with their metadata:

```
@agent_registry.register(
    name="RequirementAgent",
    version="1.0",
    skills=["AnalyzeRequirementSkill", "CompletenessCheckSkill",
            "DomainContextSkill", "ClarificationGeneratorSkill"],
    model_tier="gpt4_class",
    max_retries=2,
    timeout_seconds=60
)
class RequirementAgent(BaseAgent):
    ...
```

Agents can be hot-reloaded without restarting the application (Phase 2).

SKILL REGISTRY

Skills are reusable units of composed behavior. A skill orchestrates one or more tool calls and contains the prompt logic for LLM-involved operations.

```
@skill_registry.register(
    name="AnalyzeRequirementSkill",
    tools=["JiraTool", "KnowledgeSearchTool", "LLMTool"],
    cacheable=False
)
class AnalyzeRequirementSkill(BaseSkill):
    async def execute(self, context: SkillContext) -> SkillResult:
        ticket = await JiraTool().fetch(context.ticket_id)
        domain_docs = await KnowledgeSearchTool().search(ticket.description)
        analysis = await LLMTool().complete(
            prompt=prompts.analyze_requirement(ticket, domain_docs),
            model_tier="gpt4_class"
        )
        return SkillResult(data=analysis, evidence=[ticket.id, *domain_docs.ids])
```

TOOL REGISTRY

Tools are stateless, isolated wrappers around external systems or capabilities. Each tool:
- Accepts a typed input
- Returns a typed output
- Logs its invocation and result
- Runs in an isolated execution context (see Section 13)
- Has a configurable timeout and retry policy

```
@tool_registry.register(name="JiraTool", isolation="process")
class JiraTool(BaseTool):
    async def fetch(self, ticket_id: str) -> JiraTicket: ...
    async def search(self, query: str) -> List[JiraTicket]: ...
    async def assign(self, ticket_id: str, user_id: str) -> bool: ...
```

PLUGIN SYSTEM

New testing tools, memory backends, or external integrations are added as plugins without modifying core files:

```
backend/
  plugins/
    appium_plugin/       ← Phase 4: mobile testing
    playwright_plugin/   ← optional alternative to Selenium
    nemo_guardrails/     ← Phase 3: output safety layer
    confluence_rag/      ← Phase 2: knowledge retrieval
```

Each plugin registers its tools, skills, and agents with their respective registries on startup.

EVENT BUS

The Event Bus (Redis Pub/Sub) handles lightweight internal notifications between components. It is distinct from Celery (see Section 12).

Events published to the bus:

```
ticket.loaded
requirements.analyzed
coverage.planned
testcases.generated
testdata.resolved
automation.generated
validation.passed
validation.failed
approval.pending        ← triggers dashboard notification to engineer
approval.received
approval.rejected
execution.started
execution.completed
investigation.completed
report.generated
memory.archived
healing.suggested       ← triggers human approval for locator fix
```

EXECUTION WORKERS

Celery workers are responsible for long-running, compute-intensive tasks. Each worker:
- Picks up a task from the queue
- Creates an isolated execution environment (Docker container or process)
- Executes the task
- Reports status back via WebSocket to the dashboard
- Cleans up regardless of outcome

Worker types:
- `agent_worker` — handles LLM agent calls (generation, analysis)

- `execution_worker` — handles Robot Framework test execution
- `memory_worker` — handles embedding and vector store updates

---

8. Inter-Agent Context Protocol — TestContext

The `TestContext` is the single shared state object that flows through the entire LangGraph graph. Every agent reads from it and writes to its own designated section. No agent modifies another agent's section.

Full Schema

```
{
  "context_id": "550e8400-e29b-41d4-a716-446655440000",
  "schema_version": "1.1",
  "created_at": "2025-09-14T10:22:00Z",
  "updated_at": "2025-09-14T14:45:00Z",
  "created_by": "engineer_001",
  "workflow_status": "investigation_complete",

  "ticket": {
    "id": "JIRA-123",
    "title": "Money Transfer Feature",
    "description": "As a customer, I want to transfer money...",
    "acceptance_criteria": [
      "Transfer completes within 3 seconds",
      "Balance updates immediately",
      "Confirmation email sent within 1 minute"
    ],
    "priority": "high",
    "labels": ["banking", "payments", "sprint-24"],
    "assignee": "qa_engineer_001",
    "source": "jira",
    "raw_url": "https://company.atlassian.net/browse/JIRA-123"
  },

  "requirement_analysis": {
    "business_action": "Transfer Money",
    "domain": "banking",
    "actor": "authenticated customer",
    "preconditions": [
      "User is authenticated",
      "Source account exists and is active",
      "Account has sufficient balance"
    ],
    "expected_results": [
      "Transfer recorded in transaction history",
      "Source balance decremented by transfer amount",
      "Destination balance incremented",
      "Confirmation notification sent"
    ],
    "completeness_checklist": {
      "preconditions_defined": true,
      "actor_identified": true,
```

```
    "expected_outcome_specified": true,
    "error_scenarios_mentioned": false,
    "data_constraints_defined": false,
    "performance_expectations_set": true
  },
  "missing_fields": [
    "Transfer limit not specified",
    "Currency conversion scope not mentioned",
    "Error message wording not defined"
  ],
  "clarification_questions": [
    "What is the maximum single transfer amount?",
    "Are international/cross-currency transfers in scope?",
    "What error message should display for insufficient funds?"
  ],
  "domain_context_injected": [
    "banking_glossary_v2.md",
    "transfer_api_spec_v1.yaml",
    "payment_standards_doc.pdf"
  ],
  "memory_refs_used": ["ctx-uuid-similar-payment-ticket"]
},

"coverage_plan": {
  "risk_level": "high",
  "business_criticality": 9,
  "test_types_required": ["functional", "negative", "boundary", "performance"],
  "coverage_matrix": {
    "REQ-001 (valid transfer)": ["TC001", "TC004"],
    "REQ-002 (insufficient funds)": ["TC002"],
    "REQ-003 (invalid destination)": ["TC003"],
    "REQ-004 (transfer limit)": ["TC005"],
    "REQ-005 (concurrent transfer)": ["TC006"]
  },
  "regression_tests_to_rerun": ["TC_REG_001", "TC_REG_002"],
  "estimated_automation_effort": "medium",
  "prioritization_order": ["TC001", "TC002", "TC003", "TC005", "TC004", "TC006"]
},

"test_cases": [
  {
    "id": "TC001",
    "title": "Valid Transfer — Happy Path",
    "type": "functional",
    "priority": "critical",
    "requirement_refs": ["REQ-001"],
    "preconditions": ["User authenticated", "Balance >= transfer amount"],
    "steps": [
      "Login as authenticated user",
      "Navigate to Transfer Funds page",
      "Enter valid destination account",
      "Enter amount within balance",
      "Submit transfer",
      "Verify confirmation screen"
    ],
```

```
      "expected_outcome": "Transfer completes, balance decremented, history updated",
      "test_data_requirements": {
        "users": ["authenticated_user_with_balance"],
        "accounts": ["source_account_balance_1000_usd", "valid_destination_account"]
      }
    },
    {
      "id": "TC002",
      "title": "Insufficient Funds",
      "type": "negative",
      "priority": "high",
      "requirement_refs": ["REQ-002"],
      "preconditions": ["User authenticated", "Balance < transfer amount"],
      "steps": [
        "Login as user with low balance",
        "Attempt transfer exceeding balance",
        "Verify error message"
      ],
      "expected_outcome": "Error shown, no transfer recorded, balance unchanged",
      "test_data_requirements": {
        "users": ["authenticated_user_low_balance"],
        "accounts": ["source_account_balance_10_usd", "any_destination_account"]
      }
    }
  ],

  "test_data": {
    "TC001": {
      "strategy": "factory",
      "user": {
        "username": "factory_user_tc001",
        "password_ref": "vault://test-credentials/factory-user/password",
        "permissions": ["transfer", "view_balance", "view_history"]
      },
      "source_account": {
        "balance": 1000,
        "currency": "USD",
        "account_type": "checking"
      },
      "destination_account": {
        "id": "ACC-DEST-998",
        "type": "valid_internal"
      },
      "teardown": [
        "revert_transfer_by_reference",
        "delete_factory_user_tc001",
        "reset_account_balance_tc001"
      ]
    },
    "TC002": {
      "strategy": "fixture",
      "fixture_file": "data/JIRA-123/TC002_insufficient_funds.yaml",
      "teardown": ["delete_fixture_user_tc002"]
    }
  },
```

```
"automation": {
  "TC001": {
    "robot_file": "tests/JIRA-123/TC001_valid_transfer.robot",
    "resource_files": [
      "resources/banking_keywords.resource",
      "resources/common_keywords.resource",
      "resources/variables.resource"
    ],
    "syntax_valid": true,
    "keyword_check_passed": true,
    "library_check_passed": true,
    "data_reference_check_passed": true,
    "dry_run_passed": true,
    "validation_attempts": 1,
    "git_branch": "aegis/JIRA-123",
    "git_pr_url": "https://github.com/org/repo/pull/42"
  }
},

"execution": {
  "run_id": "exec-uuid-001",
  "triggered_by": "dashboard",
  "triggered_by_user": "engineer_001",
  "environment": "staging",
  "started_at": "2025-09-14T14:00:00Z",
  "completed_at": "2025-09-14T14:23:00Z",
  "status": "completed_with_failures",
  "results": {
    "total": 6,
    "passed": 4,
    "failed": 2,
    "skipped": 0,
    "failed_tests": ["TC001", "TC003"]
  },
  "artifacts": {
    "robot_report_html": "artifacts/exec-uuid-001/report.html",
    "junit_xml": "artifacts/exec-uuid-001/output.xml",
    "screenshots": {
      "TC001": "artifacts/exec-uuid-001/TC001_failure.png",
      "TC003": "artifacts/exec-uuid-001/TC003_failure.png"
    },
    "execution_log": "artifacts/exec-uuid-001/execution.log",
    "application_log": "artifacts/exec-uuid-001/app.log"
  }
},

"evidence": {
  "TC001": {
    "signals_detected": [
      {
        "type": "http_status_code",
        "value": "401",
        "timestamp": "2025-09-14T14:23:07Z",
        "weight": 30
```

```
      },
      {
        "type": "timestamp_correlation",
        "description": "Token expired 6 seconds before failure",
        "token_expiry": "2025-09-14T14:23:01Z",
        "failure_time": "2025-09-14T14:23:07Z",
        "gap_seconds": 6,
        "weight": 25
      },
      {
        "type": "log_keyword",
        "keyword": "TokenExpiredException",
        "log_file": "app.log",
        "line": 1847,
        "weight": 15
      }
    ],
    "confidence_score": 64,
    "confidence_calculation": "70 / 110 = 63.6%",
    "root_cause": "Authentication token expired before test execution completed",
    "recommendation": "Add token refresh step to test setup keywords or extend token TTL in staging auth c
    "similar_past_failures": ["fail-uuid-auth-001", "fail-uuid-auth-003"],
    "similar_failure_match_score": 0.91
  }
},

"reports": {
  "technical": {
    "generated_at": "2025-09-14T14:44:00Z",
    "content_path": "reports/exec-uuid-001/technical.md"
  },
  "executive": {
    "generated_at": "2025-09-14T14:44:30Z",
    "content_path": "reports/exec-uuid-001/executive.md"
  }
},

"memory_refs": {
  "similar_tickets_retrieved": [
    { "id": "ctx-uuid-payment-001", "score": 0.94 },
    { "id": "ctx-uuid-payment-003", "score": 0.87 }
  ],
  "similar_failures_retrieved": [
    { "id": "fail-uuid-auth-001", "score": 0.91 },
    { "id": "fail-uuid-auth-003", "score": 0.83 }
  ],
  "similar_scripts_retrieved": [
    { "id": "robot-uuid-banking-001", "score": 0.89 }
  ],
  "known_patterns_retrieved": [
    { "id": "pattern-uuid-token-expiry", "score": 0.96 }
  ]
},

"artifacts": {
```

```
    "robot_files": ["tests/JIRA-123/TC001_valid_transfer.robot"],
    "resource_files": ["resources/banking_keywords.resource"],
    "fixture_files": ["data/JIRA-123/TC002_insufficient_funds.yaml"],
    "git_pr_url": "https://github.com/org/repo/pull/42",
    "reports": {
      "technical": "reports/exec-uuid-001/technical.md",
      "executive": "reports/exec-uuid-001/executive.md"
    },
    "execution_artifacts": {
      "html_report": "artifacts/exec-uuid-001/report.html",
      "junit_xml": "artifacts/exec-uuid-001/output.xml"
    }
  }
}
```

Python Type Definition

```
from pydantic import BaseModel, Field
from typing import Optional, List, Dict, Any
from datetime import datetime

class TestContext(BaseModel):
    context_id: str = Field(default_factory=lambda: str(uuid4()))
    schema_version: str = "1.1"
    created_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: datetime = Field(default_factory=datetime.utcnow)
    created_by: str
    workflow_status: str = "initialized"

    ticket: Optional[TicketData] = None
    requirement_analysis: Optional[RequirementAnalysis] = None
    coverage_plan: Optional[CoveragePlan] = None
    test_cases: List[TestCase] = []
    test_data: Dict[str, TestDataBlock] = {}
    automation: Dict[str, AutomationBlock] = {}
    execution: Optional[ExecutionResult] = None
    evidence: Dict[str, EvidenceBlock] = {}
    reports: Optional[ReportBlock] = None
    memory_refs: Optional[MemoryRefs] = None
    artifacts: Optional[ArtifactBlock] = None
```

---

9. Hermes-Inspired Memory Architecture

Memory Layers

The platform implements five distinct memory layers, each with a different purpose, lifespan, and retrieval mechanism.

LAYER 1 — SHORT-TERM MEMORY (SESSION)

Scoped to a single LangGraph execution. Automatically cleared on completion. Used for: intermediate agent reasoning, prompt chain context, tool call results within a single node execution.

Storage: In-memory (Python dict within the running process)
Lifespan: Single workflow run
Retrieval: Direct access, no embedding needed

LAYER 2 — WORKFLOW MEMORY (TESTCONTEXT SNAPSHOTS)

Snapshots of the TestContext are saved at each major node completion. This enables workflow resumption after failure and audit trail reconstruction.

Storage: PostgreSQL (`workflow_snapshots` table)
Lifespan: 90 days (configurable)
Retrieval: By `context_id` or `ticket_id`, direct SQL query

LAYER 3 — EPISODIC MEMORY (PAST EXECUTIONS)

Structured summaries of completed workflows: what ticket was analyzed, what tests were generated, what failures occurred, what investigations concluded, what was archived. This is the primary source for "similar past failures" retrieval.

Storage: PostgreSQL + pgvector (embedded summaries for semantic search)
Lifespan: 1 year, then archived to cold storage
Retrieval: Semantic search on failure description, ticket title, domain tags

LAYER 4 — SEMANTIC MEMORY (DOCUMENTATION AND DOMAIN KNOWLEDGE)

Domain documentation, API specifications, Confluence pages, QA standards, and coding standards. This is the RAG knowledge base (see Section 10).

Storage: pgvector with document chunks
Lifespan: No TTL — refreshed when source documents are updated
Retrieval: Semantic search via RAG pipeline

LAYER 5 — SKILL MEMORY (PATTERNS AND FIXES)

Successful investigation patterns, approved locator fixes, test data strategies that worked, Robot Framework keyword idioms. This is what makes the system improve over time.

Storage: pgvector with structured pattern objects
Lifespan: 180 days with confidence decay
Retrieval: Semantic search on failure type, element description, domain

Memory Freshness Model

Stale memories produce confidently wrong suggestions. Every memory entry carries freshness metadata:

```
{
  "memory_id": "pattern-uuid-token-expiry",
  "type": "skill_memory",
  "content_embedding": "[vector...]",
  "created_at": "2025-03-14T09:00:00Z",
  "last_accessed": "2025-09-10T14:23:00Z",
  "access_count": 12,
  "base_confidence": 0.95,
  "ttl_days": 180,
  "is_flagged": false,
  "flagged_by": null,
  "tags": ["authentication", "token", "staging"]
}
```

Freshness Score Calculation:

```
recency_weight = exp(-days_since_creation / half_life_days)

Half-life by memory type:
  - Skill memory (locator fixes): 60 days
  - Skill memory (investigation patterns): 90 days
  - Episodic memory: 180 days
  - Semantic memory: refreshed on document update

effective_confidence = base_confidence × recency_weight × log(1 + access_count)

Retrieval ranking = cosine_similarity × effective_confidence
```

Expiry Policy:
- Entries with `effective_confidence < 0.3` are flagged for review, not auto-deleted
- Engineers can mark entries as invalid via the dashboard (sets `is_flagged = true`)
- Flagged entries are excluded from retrieval but preserved for audit
- Entries older than `ttl_days × 2` are moved to cold storage

Memory Reference Schema

When the Investigation Agent retrieves a similar past failure, the reference stored in `TestContext.memory_refs` looks like:

```
{
  "id": "fail-uuid-auth-001",
  "type": "episodic",
  "score": 0.91,
  "retrieved_at": "2025-09-14T14:23:10Z",
  "summary": "Authentication token expiry caused HTTP 401 during payment transfer test",
  "resolution": "Added token refresh keyword to Banking_Setup in resources/banking_keywords.resource",
  "context_id_origin": "ctx-uuid-payment-001",
  "effective_confidence": 0.87
}
```

ArchiveMemory Node

After every completed workflow, the `ArchiveMemory` node:

1. Embeds the `requirement_analysis` summary and stores in episodic memory
2. Embeds each `evidence` block and stores as episodic/skill memory
3. Embeds approved `.robot` script snippets and stores as skill memory
4. Updates freshness scores for any memory entries accessed during the workflow
5. Publishes `memory.archived` event to the Event Bus

---

10. Knowledge Retrieval — RAG Pipeline

RAG Indexes

The platform maintains the following vector indexes, each refreshed on a schedule or triggered by document updates:

Index	Source	Refresh Trigger	Chunk Strategy
`jira_tickets`	Jira API	On ticket update	Full ticket per chunk
`confluence_docs`	Confluence API	Daily	512-token sliding window
`api_specifications`	OpenAPI/Swagger files	On file change in Git	Per-endpoint chunk
`robot_resources`	Git repository	On merge to main	Per-keyword chunk
`qa_standards`	Uploaded documents	Manual	512-token sliding window
`failure_history`	Episodic memory store	After each ArchiveMemory	Per-investigation chunk
`git_history`	Git commit messages + diffs	On commit	Per-commit chunk

RAG Pipeline

```
User Query or Agent Need
        |
        ▼
   Embed Query
   (text-embedding-3-small or local equivalent)
        |
        ▼
   Vector Search
   (pgvector cosine similarity, top-k=20)
        |
        ▼
   Re-Rank Results
   (cross-encoder/ms-marco-MiniLM-L-6-v2)
   Returns top-k=5 with precision scores
        |
        ▼
   Filter by Freshness Score
   (exclude entries with effective_confidence < 0.3)
        |
        ▼
   Inject into Agent Prompt
   (as grounded references with source attribution)
```

Re-Ranker Decision

The re-ranker is `cross-encoder/ms-marco-MiniLM-L-6-v2` — a small (22M parameter) cross-encoder model that runs locally via Ollama or as a standalone process. It was chosen because:

- Precision matters more than recall for a QA platform (wrong suggestions are costly)
- It is small enough to run locally without GPU
- It has strong performance on passage relevance ranking benchmarks
- It adds ~50ms latency per re-ranking batch, which is acceptable for async agent calls

Vector Database Decision

Decision: Start with pgvector. Migrate to Qdrant if scale requires it.

Criterion	pgvector	Qdrant
Infrastructure	Same PostgreSQL instance	Additional service
Query latency (< 500K vectors)	~10–30ms	~5–15ms
Query latency (> 1M vectors)	Degrades	Remains fast
Operational complexity	Low (one DB)	Medium (separate service)
ACID transactions with other data	Yes	No
Migration effort	—	Medium

For a QA platform with bounded embedding volume (tickets, scripts, failure summaries), pgvector handles the load comfortably up to ~500K vectors. A migration trigger is defined: if the total vector count exceeds 500K or average query latency exceeds 200ms, migrate to Qdrant. The migration is abstracted behind a `VectorStore` interface so the swap is a configuration change, not a code rewrite.

```
# Vector store abstraction
class VectorStore(Protocol):
    async def embed_and_store(self, text: str, metadata: dict) -> str: ...
    async def search(self, query: str, top_k: int, filters: dict) -> List[SearchResult]: ...
    async def delete(self, memory_id: str) -> bool: ...

# Current implementation
class PgVectorStore(VectorStore): ...

# Future swap — no agent or skill code changes needed
class QdrantStore(VectorStore): ...
```

---

11. AI Model Strategy

Model Tier Assignments

Not all tasks require the same reasoning capability. Assigning model tiers by task type controls both cost and reliability.

Task	Agent	Required Tier	Reasoning
Requirement analysis	RequirementAgent	GPT-4 class	Business logic reasoning, ambiguity detection in vague acceptance criteria
Coverage planning	CoveragePlanner	GPT-4 class	Risk assessment requires judgment about business criticality
Test case generation	TestCaseGenerator	GPT-4 class	Creative coverage analysis, edge case identification
Robot Framework code generation	AutomationGenerator	GPT-4 class	Syntactically precise, structured output; local models fail here
Test data strategy selection	TestDataResolver	GPT-3.5 class	Template selection from known strategies, lower reasoning load
Failure investigation	InvestigationCoordinator	GPT-4 class	Multi-signal causal reasoning across heterogeneous evidence
Locator candidate ranking	LocatorRepairAgent	Local (Ollama)	Embedding similarity scoring, no reasoning required
Evidence signal classification	InvestigationCoordinator	Local (Ollama)	Binary classification (signal present / absent) from structured logs
Technical report generation	ReportGenerator	GPT-3.5 class	Templated summarization of structured evidence data
Executive summary generation	ReportGenerator	GPT-3.5 class	Same — structured input, templated narrative output
Re-ranking RAG results	All agents	Local (ms-marco)	Cross-encoder ranking, no LLM reasoning
Memory embedding	ArchiveMemory	Local (embedding model)	text-embedding-3-small or nomic-embed-text via Ollama

Model Provider Configuration

```
# backend/config/models.yaml
providers:
  gpt4_class:
    primary: openai/gpt-4o
    fallback: anthropic/claude-opus-4
    timeout_seconds: 60
  gpt35_class:
    primary: openai/gpt-4o-mini
    fallback: ollama/llama3.1:8b
    timeout_seconds: 30
  local:
    embedding: ollama/nomic-embed-text
    reranker: local/ms-marco-MiniLM-L-6-v2
    classification: ollama/llama3.1:8b
    timeout_seconds: 15
```

Cost Estimate

At moderate usage (10 tickets/day, average 5 test cases each, 3 failures/day requiring investigation):

Operation	Calls/Day	Avg Tokens/Call	Estimated Cost/Day
Requirement analysis	10	~2,000	~$0.10
Coverage planning	10	~1,500	~$0.08
Test case generation	50	~1,800	~$0.40
Robot Framework generation	50	~2,500	~$0.60
Failure investigation	15	~3,000	~$0.40
Report generation (GPT-3.5)	10	~2,000	~$0.04
Local tasks	—	—	$0.00
Daily Total			**~$1.62/day**

At 20 business days/month: approximately **$32/month** in API costs at this usage level.

---

12. Event Bus vs Celery — Roles and Boundaries

These are complementary, not competing, systems. Their roles are distinct:

Dimension	Celery + Redis	Event Bus (Redis Pub/Sub)
Purpose	Distribute and execute long-running tasks across workers	Broadcast lightweight notifications within the application
Payload size	Large (full agent context, test suite)	Small (event type, IDs, status)
Persistence	Tasks queued durably until consumed	Fire-and-forget; not persisted
Consumers	One worker picks up and processes the task	Multiple services can subscribe to same event
Latency expectation	Seconds to minutes	Milliseconds
Examples	Execute test suite, run LLM agent, embed documents	"Approval pending" → notify dashboard, "Execution complete" → trigger investigation

```
# Celery: dispatch heavy work
celery_app.send_task(
    "workers.execution.run_suite",
    args=[context_id, suite_config],
    queue="execution"
)

# Event Bus: notify listeners
await event_bus.publish("execution.completed", {
    "context_id": context_id,
    "status": "completed_with_failures",
    "failed_tests": ["TC001", "TC003"]
})
```

---

13. Tool Isolation Model

"Tools are isolated" is a design principle that requires an actual isolation mechanism for each tool type. Insufficient isolation risks cross-test contamination, credential leakage, and unintended side effects.

Tool	Isolation Mechanism	Reason
BrowserTool	Docker container per test	Selenium sessions must not share browser state
DatabaseTool (read)	Read-only database user	Investigation queries cannot modify data
DatabaseTool (write)	Separate write user, scoped to test schema	Test data setup cannot touch production tables
FilesystemTool	chroot to designated work directory	Agent cannot read/write outside its sandbox
DockerTool	Docker-in-Docker (DinD) with resource limits	Robot Framework execution must not affect host
SSHTool	Restricted keypair with no sudo rights	Log retrieval only
VaultTool	Short-lived tokens, one secret per request	Credentials not held in memory between calls
LLMTool	Process-level isolation with timeout	Prevents infinite loops and runaway token usage
JiraTool / AzureTool	OAuth 2.0 scoped tokens	Read-only by default; write only for assignment

All tool invocations are logged with: `tool_name`, `input_hash`, `output_hash`, `duration_ms`, `isolation_type`, `caller_skill`, `caller_agent`, `context_id`. This forms the tool audit trail.

---

14. Module Specifications

Module 1 — Dashboard & Ticket Management

Responsibilities:
- User authentication (JWT, role-based: Admin / QA Engineer / Viewer)
- Dashboard overview: pending approvals count, active executions, recent failures, memory health
- Ticket browser: list tickets from connected sources with filter by project, priority, assignee, status
- Ticket search across Jira, Azure DevOps, GitLab Issues
- Ticket assignment to QA engineers
- Execution history with filterable results (by ticket, by environment, by status, by date range)
- Real-time execution status via WebSocket
- Memory management panel: view flagged entries, manually invalidate stale memories
- Audit log viewer (all AI-generated actions, approvals, commits)

Integrations: Jira (OAuth 2.0), Azure DevOps (PAT), GitLab Issues (OAuth 2.0)

Module 2 — Requirement Analysis Agent

Node: `RequirementAgent`
Skills: `AnalyzeRequirementSkill`, `CompletenessCheckSkill`, `DomainContextSkill`, `ClarificationGeneratorSkill`

Workflow:

```
1. Fetch ticket via JiraTool
2. Retrieve similar past tickets from episodic memory (semantic search)
3. Retrieve domain documentation via KnowledgeSearchTool (RAG)
4. Run CompletenessCheckSkill against standard checklist
5. Run AnalyzeRequirementSkill with: ticket + domain docs + similar tickets
6. Generate clarification questions for every unchecked completeness item
7. Write to TestContext.requirement_analysis
```

Completeness Checklist (evaluated for every ticket):
- [ ] Actor / subject identified
- [ ] Preconditions defined
- [ ] Expected outcome specified
- [ ] Error / failure scenarios mentioned
- [ ] Data inputs and constraints defined
- [ ] Performance expectations stated

Any unchecked item → targeted clarification question generated. The agent asks; it does not invent answers to unanswered questions.

Domain Context Injection:
Engineers can upload domain documents (API specs, glossaries, standards) to the platform. These are chunked, embedded, and stored in the `semantic_memory` RAG index. The Requirement Agent retrieves the top-5 most relevant chunks per ticket before analysis, grounding its output in actual domain knowledge rather than LLM-internal assumptions.

Example:

Input:

```
As a customer,
I want to transfer money
so that I can send funds to another account.
```

Output:

```
Business Action: Transfer Money
Domain: Banking (inferred from domain context)
Actor: Authenticated customer

Preconditions:
- User is authenticated
- Source account is active and has sufficient balance

Expected Results:
- Transfer recorded in transaction history
- Source balance decremented
- Destination balance incremented
- Confirmation notification dispatched

Completeness Gaps:
- Error scenarios not mentioned
- Data constraints not defined (transfer limit, currency)

Clarification Questions:
1. What is the maximum single transfer amount?
2. What error message should display for insufficient funds?
3. Are international transfers in scope?

Similar Tickets Retrieved:
- JIRA-089 (Payment Feature, score: 0.94) — prior analysis available
```

Module 3 — Coverage Planner

Node: `CoveragePlanner`
Skills: `RiskAssessmentSkill`, `CoverageMatrixSkill`, `RegressionSelectorSkill`, `PrioritizationSkill`

This node was absent in v2 and is a meaningful addition. It sits between Requirement Analysis and Test Case Generation and performs four functions:

Risk Assessment:
Evaluates the business criticality of the feature being tested. Inputs: ticket priority, domain (payments > UI cosmetics), number of integration points, past failure rate for similar tickets. Output: risk level (low/medium/high/critical) and a numerical business criticality score (1–10).

Coverage Matrix:
Maps each identified requirement to the test types it needs. A payment transfer needs functional, negative, boundary, and at least a basic performance check. A UI label change needs only functional and a quick regression.

Regression Selection:
Queries episodic memory for existing tests related to this domain. If a "transfer funds" test suite already exists from a prior ticket, those tests are flagged for re-execution alongside the new tests. This prevents the platform from generating duplicate tests for features that were already tested.

Prioritization:
Orders test cases by `risk_score × coverage_impact`. Critical path tests execute first. This matters for CI pipelines with time budgets.

Module 4 — Test Case Generation Agent

Node: `TestCaseGenerator`
Skills: `FunctionalTestSkill`, `NegativeTestSkill`, `BoundaryTestSkill`, `RegressionMarkerSkill`

Generates test cases in four categories:
- **Functional** — happy path, primary user journey
- **Negative** — invalid inputs, constraint violations, unauthorized access
- **Boundary** — minimum/maximum values, empty inputs, maximum-length strings
- **Regression** — markers linking to existing tests that should re-run

Each test case declares its `test_data_requirements` in structured form, feeding directly into Module 5.

Coverage Matrix Output (example):

```
TC001  Valid Transfer             functional   REQ-001  critical
TC002  Insufficient Funds         negative     REQ-002  high
TC003  Invalid Destination        negative     REQ-003  high
TC004  Zero Amount Transfer       boundary     REQ-001  medium
TC005  Maximum Limit Transfer     boundary     REQ-004  high
TC006  Concurrent Transfer        edge         REQ-005  medium
```

Module 5 — Test Data Management

Node: `TestDataResolver`
Skills: `DataDeclarationSkill`, `FactoryResolutionSkill`, `FixtureResolutionSkill`, `DatabaseSeedSkill`, `TeardownPlannerSkill`

This is one of the most important modules in the system. Test automation fails in practice not because the scripts are wrong but because the data isn't there, isn't in the right state, or leaves the environment dirty.

Resolution Strategies:

Strategy	Description	When to Use
`factory`	Generate data programmatically via Faker/factory_boy	Isolated tests needing clean, controlled data
`fixture`	Load from static YAML/JSON fixture files	Stable, predictable scenarios needing specific values
`database_seed`	Execute SQL/NoSQL seed scripts before run	Integration tests requiring real database relationships
`api_bootstrap`	Call setup endpoints to create entities via API	End-to-end flows that need entities created through the application
`masked_production`	Anonymized snapshots of real data	Realistic edge cases, data-shape testing

Strategy is selected per test case. Engineers can override the agent's selection in the Human Approval layer.

Credential Management:
No credential is stored in a `.robot` file, a fixture file, or the PostgreSQL database in plaintext. Credentials are stored in HashiCorp Vault and referenced via Vault path:

```
vault://test-credentials/staging/db-password
vault://test-credentials/factory-user/password
vault://api-keys/jira/service-account
```

The `VaultTool` fetches credentials at runtime with a short-lived token. The token is never logged.

Teardown Strategy:
Every test data block declares a cleanup action. The execution engine runs teardown unconditionally after each test — whether the test passes, fails, or is skipped. No test leaves the environment dirty.

```
teardown:
  - action: revert_transfer_by_reference
    params: { reference_field: "confirmation_number" }
  - action: delete_factory_user
    params: { username_field: "user.username" }
  - action: reset_account_balance
    params: { account_field: "source_account.id", target_balance: 1000 }
```

Module 6 — Automation Generation Agent

Node: `AutomationGenerator`
Skills: `RobotFrameworkSkill`, `SeleniumKeywordSkill`, `APIKeywordSkill`, `ResourceFileSkill`

Converts test cases and their resolved data blocks into executable Robot Framework files.

Generated Output Structure:

```
tests/
  JIRA-123/
    TC001_valid_transfer.robot
    TC002_insufficient_funds.robot
    TC003_invalid_destination.robot
resources/
  banking_keywords.resource    ← domain-specific reusable keywords
  common_keywords.resource     ← shared keywords across domains
  variables.resource           ← environment variables, base URLs
data/
  JIRA-123/
    TC002_insufficient_funds.yaml   ← fixture data
```

Validation Pipeline (before human review):

```
Generated .robot file
        |
        ▼
Step 1: Syntax Check
        robot --dryrun tests/JIRA-123/TC001_valid_transfer.robot
        |
        ├── FAIL → inject error into agent prompt, retry (max 2x)
        ▼
Step 2: Keyword Existence Check
        Verify all called keywords are defined in resource files or standard libraries
        |
        ├── FAIL → identify missing keywords, retry with specific instruction
        ▼
Step 3: Library Import Check
        Verify all required libraries (SeleniumLibrary, RequestsLibrary, etc.) are imported
        |
        ├── FAIL → add missing import, retry
        ▼
Step 4: Data Reference Check
        Verify all ${VARIABLE} references exist in variables.resource or TestContext.test_data
        |
        ├── FAIL → retry
        ▼
Step 5: Pass → Queue for Human Review
        FAIL (after 2 retries on any step) → Flag for manual intervention
```

The memory agent retrieves similar approved scripts from the skill memory index. These are injected into the AutomationGenerator's prompt as examples, improving output quality on repeated similar tickets.

Module 7 — Human Validation Layer

Node: `HumanApproval`

Review Interface:
- Side-by-side view: original ticket + generated test case + generated `.robot` file
- Inline editing of `.robot` files directly in the browser
- Per-test approve / request changes / reject
- Comment and annotation field (stored in audit log)
- Version diff between original and edited versions
- Keyboard shortcut support for rapid review

Approval Actions:
- **Approve** → triggers Git commit + PR creation + execution queue
- **Request Changes** → sends back to `AutomationGenerator` with reviewer's comments injected into the agent's next prompt
- **Reject with regenerate test cases** → sends back to `TestCaseGenerator` (wrong test scope)
- **Reject and close** → closes the workflow, logs reason

Git Integration — on Approval:

```
Engineer approves TC001 →
  1. Create branch: aegis/JIRA-123
  2. Commit files:
     - tests/JIRA-123/TC001_valid_transfer.robot
     - resources/banking_keywords.resource (if new keywords added)
     - data/JIRA-123/ (if fixtures generated)
  3. Commit message: "aegis: JIRA-123 — TC001 Valid Transfer [approved by engineer_001]"
  4. Open Pull Request:
     - Title: "[AegisQA] JIRA-123 — Money Transfer Test Suite"
     - Body: ticket summary + test case list + coverage matrix
  5. Store PR URL in TestContext.artifacts.git_pr_url
  6. Publish approval.received event
```

Commits are attributed to the approving engineer's Git identity, not a system account. The history reflects who reviewed and approved each generated test.

Module 8 — Execution Engine

Node: `ExecutionDispatcher`
Skills: `SuiteExecutionSkill`, `ResultCollectionSkill`, `ArtifactStorageSkill`

Technologies:
- Robot Framework
- Selenium WebDriver (Docker-isolated browser container)
- RequestsLibrary (API tests)
- Celery + Redis (async task dispatch)
- Docker (isolated execution environments)

Concurrent Execution Architecture:

Each execution run is dispatched as an isolated Celery task. Each task:
- Runs in its own Docker container with a dedicated browser session
- Has its own log and output directory under `artifacts/{run_id}/`
- Reports progress back to the dashboard via WebSocket in real time
- Runs teardown unconditionally after test completion
- Cannot share resources or state with concurrent runs

Execution Triggers:

Dashboard (manual):

```
Engineer clicks "Run Suite" →
  POST /api/v1/execute
  { "context_id": "...", "env": "staging" }
```

CI/CD Webhook:

```
# Trigger from GitHub Actions, Jenkins, GitLab CI, Azure DevOps
curl -X POST https://aegisqa.internal/api/v1/execute \
  -H "Authorization: Bearer ${AEGISQA_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{
    "suite": "regression",
    "branch": "feature/JIRA-123",
    "env": "staging",
    "tags": ["JIRA-123"]
  }'
```

GitHub Actions integration:

```
jobs:
  aegisqa:
    runs-on: ubuntu-latest
    steps:
      - name: Trigger AegisQA Suite
        run: |
          RUN_ID=$(curl -s -X POST ${{ secrets.AEGISQA_URL }}/api/v1/execute \
            -H "Authorization: Bearer ${{ secrets.AEGISQA_TOKEN }}" \
            -d '{"suite": "smoke", "env": "staging"}' | jq -r .run_id)
          echo "Run ID: $RUN_ID"

      - name: Wait for Results
        run: |
          curl -s ${{ secrets.AEGISQA_URL }}/api/v1/results/$RUN_ID/junit.xml \
            > test-results.xml

      - name: Publish Results
        uses: mikepenz/action-junit-report@v4
        with:
          report_paths: test-results.xml
```

Collected Artifacts Per Run:
- Robot Framework HTML report (human-readable)
- JUnit XML (machine-readable, CI/CD compatible)
- Screenshots on failure (per failed test)
- Video recording (optional, configurable via environment settings)
- Robot Framework execution log
- Application log (if log endpoint configured in environment settings)
- HAR file (HTTP Archive) for API test runs

Module 9 — Investigation Agent

Node: `InvestigationCoordinator`
Skills: `LogAnalysisSkill`, `APIResponseAnalysisSkill`, `EvidenceScoringSkill`, `SimilarFailureRetrievalSkill`, `RootCauseSkill`

Evidence Sources (correlated per failed test):
- Robot Framework output.xml (test steps, timestamps, exact failure point)
- Application log (exceptions, warnings, stack traces)
- API response log (status codes, response times, response bodies)

- Database state snapshot (before/after comparison if configured)
- Execution log (environment setup, teardown)

Evidence-Based Confidence Scoring:

The LLM identifies and extracts evidence signals. A deterministic function scores them. No confidence number is ever generated by the LLM itself.

```
EVIDENCE_WEIGHTS = {
    "http_status_code_match": 30,     # 4xx/5xx matching known error class
    "timestamp_correlation": 25,      # Error within 30s window of failure
    "log_keyword_match": 15,          # Known exception/error keyword in logs
    "historical_pattern_match": 20,   # Same pattern seen in past failure memory
    "network_anomaly_detected": 20,   # Timeout, DNS failure, TCP reset
    "database_error_correlated": 15,  # DB exception matching failure timing
}

MAX_POSSIBLE_WEIGHT = sum(EVIDENCE_WEIGHTS.values())  # = 125

def calculate_confidence(detected_signals: List[EvidenceSignal]) -> float:
    total = sum(EVIDENCE_WEIGHTS[s.type] for s in detected_signals
                if s.type in EVIDENCE_WEIGHTS)
    return round((total / MAX_POSSIBLE_WEIGHT) * 100, 1)
```

Similar Failure Retrieval:
Before generating a root cause, the agent queries episodic memory for past failures with similar evidence patterns. If a match is found (cosine similarity > 0.80), the past resolution is surfaced alongside the current analysis.

Output Example:

```
Failed Test: TC001 — Valid Transfer

Root Cause Assessment:
Authentication token expired before the transfer request was submitted.

Evidence (Confidence: 64%):
  ✓ [+30] HTTP 401 Unauthorized returned by /api/v1/transfer
  ✓ [+25] Token expiry at 14:23:01, failure at 14:23:07 (6-second gap)
  ✓ [+15] "TokenExpiredException" found in app.log at line 1847
  ✗ [+00] No matching historical pattern found
  ✗ [+00] No network-level anomaly detected

Score: 70 / 110 = 64%

Similar Past Failure Retrieved:
  fail-uuid-auth-001 (JIRA-089, score: 0.91)
  Resolution: Added token refresh keyword to Banking_Setup in resources/banking_keywords.resource

Recommended Action:
  Add token refresh step to test setup or extend token TTL in staging auth config.
  See similar resolution in JIRA-089.
```

Module 10 — Self-Healing Agent (Locator Repair)

Node: Not in MVP — Phase 2
Skills: `LocatorExtractionSkill`, `DOMSnapshotSkill`, `CandidateGenerationSkill`, `SimilarityRankingSkill`

Detection Mechanism:
Monitors `ElementNotFoundError` and `ElementNotInteractableError` exceptions in Robot Framework output after execution. When detected:

1. Extract the failed locator from the exception
2. Capture the current page DOM via a targeted Selenium snapshot
3. Retrieve similar approved locator fixes from skill memory (embedding search)
4. Generate candidate replacement locators using LLM + DOM context
5. Score candidates: cosine similarity to original + structural proximity + attribute stability weight

Stability Heuristics:
- `data-testid` attributes → high stability (set by developers for testing)
- `id` attributes → high stability (usually stable across releases)
- `class` attributes → medium stability (often changed during UI redesigns)
- `xpath` with text content → low stability (changes with copy changes)

Output:

```
Broken Locator Detected:
  File: tests/JIRA-123/TC001_valid_transfer.robot
  Line: 24
  Strategy: id
  Value: submitBtn

Candidate Replacements:
  1. id=submitButton              similarity: 0.94  stability: high    ← RECOMMENDED
  2. css=button[type=submit]      similarity: 0.81  stability: medium
  3. xpath=//button[@data-testid='submit-transfer']
                                  similarity: 0.78  stability: high

Similar Fix Retrieved from Memory:
  pattern-uuid-locator-001 (JIRA-091)
  Applied: id=submitBtn → id=submitButton (score: 0.94)

Status: Awaiting human approval — no file has been modified.
```

Human approval required. On approval: the fix is committed to the aegis branch, PR updated, and the pattern archived in skill memory with `access_count + 1`.

Module 11 — AI Reporting Agent

Node: `ReportGenerator`
Skills: `TechnicalReportSkill`, `ExecutiveSummarySkill`

Technical Report (for QA engineers and SDETs):

```
# Test Execution Report — JIRA-123
Date: 2025-09-14 | Environment: Staging | Run ID: exec-uuid-001

## Summary
Total: 6 | Passed: 4 | Failed: 2 | Duration: 23 minutes

## Failed Tests

### TC001 — Valid Transfer
Root Cause: Authentication token expired before request submission
Confidence: 64%
Evidence:
  - HTTP 401 from /api/v1/transfer
  - TokenExpiredException in app.log:1847
  - Token expired 6s before failure timestamp
Similar Past Failure: JIRA-089 (score: 0.91)
Recommendation: Add token refresh to Banking_Setup or extend staging token TTL

### TC003 — Transfer to Invalid Destination
Root Cause: Account validation service returned 503
Confidence: 81%
Evidence:
  - HTTP 503 from /api/v1/accounts/validate
  - Network timeout at 14:12:03 (30s timeout exceeded)
  - "ServiceUnavailableException" in app.log:2203
Recommendation: Check staging account service health; may be infrastructure issue

## Artifacts
[HTML Report] [JUnit XML] [TC001 Screenshot] [TC003 Screenshot] [Execution Log]
```

Executive Summary (for product owners and managers):

```
# QA Status — Sprint 24 | Money Transfer Feature

Tests Run: 6   Passed: 4 (67%)   Failed: 2

## What Failed and Why

**Authentication instability in staging**
One test failed because the authentication service in staging is configured
with a very short token lifetime. This is a staging configuration issue,
not a problem with the Money Transfer feature itself.

**Account validation service was unavailable**
One test failed because a supporting service was temporarily offline in staging.
This is an infrastructure issue unrelated to the feature under test.

## Risk Assessment
No functional regressions detected in the core transfer logic.
Both failures are infrastructure/environment issues, not feature bugs.

## Recommendation
Resolve staging authentication config before final regression run.
Feature is ready for further testing once environment is stable.
```

Module 12 — Memory Agent

Node: `ArchiveMemory`
Skills: `EmbedAndStoreSkill`, `FreshnessEvaluationSkill`, `ContextArchiveSkill`

Runs after every completed workflow, regardless of success or failure.

What is archived:

Content	Memory Layer	Embedding Source
Requirement analysis summary	Episodic	`business_action` + `domain` + key preconditions
Coverage plan	Episodic	`risk_level` + `test_types` + ticket tags
Evidence blocks (per failed test)	Episodic + Skill	`root_cause` + `signals` + `recommendation`
Approved `.robot` scripts	Skill	Keyword names + resource imports + domain tags
Locator fixes (approved)	Skill	Old locator + new locator + stability score

Freshness updates:
Every memory entry accessed during the workflow (`memory_refs` in TestContext) has its `last_accessed` and `access_count` updated. This increases its retrieval ranking for future similar workflows.

Module 13 — Wireshark / Network Analysis Agent

Node: Post-MVP — Phase 3
Skills: `TsharkCaptureSkill`, `PacketCorrelationSkill`, `NetworkAnomalySkill`

Parsing raw binary pcap files is non-trivial. The Phase 3 approach uses `tshark` — the CLI companion to Wireshark — which produces structured JSON output that is tractable for an LLM agent.

Technical Prerequisites Before Building:
- Millisecond-precision timestamps in Robot Framework output (for correlation)
- Network capture scoped to test execution host (not full network traffic)
- `tshark` installed on execution worker host
- Structured JSON pipeline from tshark output to Investigation Agent

Approach:

```
# Capture during test execution
tshark -i eth0 -f "host staging.company.com" -w /tmp/capture_${RUN_ID}.pcap &

# After execution: filter and export as JSON
tshark -r /tmp/capture_${RUN_ID}.pcap \
  -T json \
  -Y "http.response.code >= 400 or tcp.analysis.retransmission" \
  > /tmp/network_events_${RUN_ID}.json
```

The resulting JSON is then correlated against Robot Framework execution timestamps to identify network-level events that coincide with test failures — timeouts, connection resets, DNS failures, unexpected responses.

---

15. Agent Catalog

Agent	Node	Primary Skills	Model Tier
RequirementAgent	`requirement_agent`	AnalyzeRequirement, CompletenessCheck, DomainContext, ClarificationGenerator	GPT-4 class
CoveragePlanner	`coverage_planner`	RiskAssessment, CoverageMatrix, RegressionSelector, Prioritization	GPT-4 class
TestCaseGenerator	`test_case_generator`	FunctionalTest, NegativeTest, BoundaryTest, RegressionMarker	GPT-4 class
TestDataResolver	`test_data_resolver`	DataDeclaration, FactoryResolution, FixtureResolution, DatabaseSeed, TeardownPlanner	GPT-3.5 class
AutomationGenerator	`automation_generator`	RobotFramework, SeleniumKeyword, APIKeyword, ResourceFile	GPT-4 class
ValidationAgent	`validator`	SyntaxCheck, KeywordExistence, LibraryImport, DataReference	Local
ExecutionAgent	`execution_dispatcher`	SuiteExecution, ResultCollection, ArtifactStorage	N/A (orchestration)
InvestigationCoordinator	`investigation_coord`	LogAnalysis, APIResponseAnalysis, EvidenceScoring, SimilarFailureRetrieval, RootCause	GPT-4 class
LocatorRepairAgent	`locator_repair`	LocatorExtraction, DOMSnapshot, CandidateGeneration, SimilarityRanking	Local (embeddings)
ReportGenerator	`report_generator`	TechnicalReport, ExecutiveSummary	GPT-3.5 class
MemoryAgent	`archive_memory`	EmbedAndStore, FreshnessEvaluation, ContextArchive	Local (embedding)

16. Skill Catalog

Requirement Agent Skills

Skill	Tools Used	Description
`AnalyzeRequirementSkill`	JiraTool, KnowledgeSearchTool, LLMTool	Extracts structured requirements from ticket text, grounded in domain context
`CompletenessCheckSkill`	LLMTool	Evaluates ticket against 6-item completeness checklist
`DomainContextSkill`	KnowledgeSearchTool, VectorDBTool	Retrieves relevant domain documentation from semantic memory
`ClarificationGeneratorSkill`	LLMTool	Produces targeted questions for each completeness gap

Coverage Planner Skills

Skill	Tools Used	Description
`RiskAssessmentSkill`	LLMTool, VectorDBTool	Scores business criticality and assigns risk level
`CoverageMatrixSkill`	LLMTool	Maps each requirement to required test types
`RegressionSelectorSkill`	VectorDBTool, DatabaseTool	Identifies existing tests to re-run
`PrioritizationSkill`	LLMTool	Orders test cases by risk × coverage impact

Test Generation Skills

Skill	Tools Used	Description
`FunctionalTestSkill`	LLMTool	Generates happy-path test cases
`NegativeTestSkill`	LLMTool	Generates constraint violation and error path tests
`BoundaryTestSkill`	LLMTool	Generates min/max/edge value tests
`RegressionMarkerSkill`	DatabaseTool, VectorDBTool	Tags tests for regression tracking

Test Data Skills

Skill	Tools Used	Description
`DataDeclarationSkill`	LLMTool	Parses test case data requirements into structured declarations
`FactoryResolutionSkill`	DatabaseTool, LLMTool	Generates factory data via Faker/factory_boy
`FixtureResolutionSkill`	FilesystemTool	Loads and validates fixture YAML/JSON files
`DatabaseSeedSkill`	DatabaseTool	Executes seed scripts against target environment
`TeardownPlannerSkill`	LLMTool	Builds cleanup action plan per test data block

Automation Skills

Skill	Tools Used	Description
`RobotFrameworkSkill`	LLMTool, VectorDBTool	Generates .robot test files
`SeleniumKeywordSkill`	LLMTool	Generates Selenium-based UI keywords
`APIKeywordSkill`	LLMTool	Generates RequestsLibrary API keywords
`ResourceFileSkill`	LLMTool, FilesystemTool	Generates and updates resource files

Validation Skills

Skill	Tools Used	Description
`SyntaxCheckSkill`	RobotTool (`--dryrun`)	Validates Robot Framework syntax
`KeywordExistenceSkill`	FilesystemTool	Verifies all called keywords are defined
`LibraryImportSkill`	FilesystemTool	Verifies all library imports are present
`DataReferenceSkill`	DatabaseTool	Verifies all variable references are resolvable

Investigation Skills

Skill	Tools Used	Description
`LogAnalysisSkill`	FilesystemTool, LLMTool	Extracts error signals from application and Robot logs
`APIResponseAnalysisSkill`	FilesystemTool, LLMTool	Correlates API responses with failure timestamps
`EvidenceScoringSkill`	— (deterministic)	Computes confidence score from detected evidence signals
`SimilarFailureRetrievalSkill`	VectorDBTool	Retrieves similar past failures from episodic memory
`RootCauseSkill`	LLMTool	Generates root cause narrative from scored evidence

Memory Skills

Skill	Tools Used	Description
`EmbedAndStoreSkill`	LLMTool (embedding), VectorDBTool	Embeds content and writes to vector store
`FreshnessEvaluationSkill`	DatabaseTool	Applies freshness decay model to accessed memories
`ContextArchiveSkill`	DatabaseTool, VectorDBTool	Writes full TestContext snapshot to episodic store

17. Tool Catalog

Tool	Isolation	Permissions	Description
`JiraTool`	Process	OAuth 2.0 read + write (assign)	Fetch, search, assign tickets
`AzureDevOpsTool`	Process	PAT read + write	Work items, pipelines
`GitTool`	Process	Branch, commit, PR (scoped repo)	Create branches, commit files, open PRs
`RobotTool`	Docker container	Execution sandbox	Run robot suites, dryrun validation
`BrowserTool`	Docker container	Target URL only	Selenium WebDriver operations
`APITool`	Process	Configurable per environment	HTTP requests via RequestsLibrary
`DatabaseTool (read)`	Process	Read-only DB user	Investigation queries
`DatabaseTool (write)`	Process	Write to test schema only	Test data setup
`FilesystemTool`	chroot sandbox	Designated work directory	Read/write test artifacts
`VaultTool`	Process	Short-lived token per request	Credential retrieval
`VectorDBTool`	Process	pgvector read/write	Embedding storage and search
`LLMTool`	Process with timeout	API key scoped	LLM completions and embeddings
`KnowledgeSearchTool`	Process	VectorDB read	Semantic search over RAG indexes
`SSHTool`	Process	Restricted keypair, no sudo	Log retrieval from remote hosts
`DockerTool`	DinD with resource limits	Scoped image list	Spin up/down test execution containers

18. Non-Functional Requirements

Performance

- Dashboard reads: < 500ms response time
- Agent tasks: asynchronous via Celery; progress updates via WebSocket
- Execution: at least 10 concurrent test suite runs (horizontally scalable via Celery workers)
- RAG retrieval: < 200ms (pgvector with HNSW index)
- Memory archiving: non-blocking, runs asynchronously after workflow completion

Security

- Authentication: JWT with short expiry (15 minutes) + refresh tokens
- Role-based access: Admin (full), QA Engineer (create, review, execute), Viewer (read-only)
- Secrets: HashiCorp Vault; never in logs, never in `.robot` files, never in database plaintext
- Tool audit trail: all tool calls logged with `input_hash`, `output_hash`, `caller`, `context_id`
- Git commits attributed to approving engineer, not system account
- Network: all LLM API calls over HTTPS; Vault accessed over mTLS

Scalability

- Celery workers are horizontally scalable (add workers without code changes)
- Agent and skill registries support hot-reload in Phase 2
- Vector store abstracted behind interface (pgvector → Qdrant migration is config change)
- Plugin architecture for new test tools, memory backends, integrations

Maintainability

- Agent prompts stored as versioned Jinja2 templates, not hardcoded strings
- Model provider is configurable per agent (swap OpenAI for Anthropic or Ollama per task)
- Every agent, skill, and tool has a corresponding unit test with mocked dependencies
- Skills are reusable across agents — no skill is owned by a single agent

Observability

- LangGraph execution traces exportable to LangSmith or local trace store
- Celery task status visible in Flower dashboard
- Application logs structured JSON (stdout) compatible with ELK, Datadog, CloudWatch
- WebSocket endpoint exposes real-time execution state to dashboard

---

19. Technology Stack

Backend

Component	Technology	Role
API Framework	FastAPI	REST + WebSocket gateway
Orchestration	LangGraph	Agent state machine
Task Queue	Celery + Redis	Async long-running tasks
Database	PostgreSQL	Primary data store
Vector Store	pgvector (Phase 1) / Qdrant (Phase 3+)	Embedding storage
Cache	Redis	Session cache, Event Bus, Celery broker
Secrets	HashiCorp Vault	Credential management

AI Layer

Component	Technology	Role
GPT-4 class	OpenAI GPT-4o / Anthropic Claude	Generation, reasoning, investigation
GPT-3.5 class	GPT-4o-mini / Llama 3.1 8B	Reports, templated tasks
Local models	Ollama	Embeddings, classification, re-ranking
Embeddings	nomic-embed-text (Ollama)	Vector embedding
Re-ranker	ms-marco-MiniLM-L-6-v2	RAG result precision

Automation

Component	Technology	Role
Test Framework	Robot Framework	Test execution
UI Automation	Selenium WebDriver	Browser-based tests
API Testing	RequestsLibrary	HTTP API tests
Test Data	Faker, factory_boy	Data generation

Infrastructure

Component	Technology	Role
Containerization	Docker + Docker Compose	All services containerized
Execution Isolation	Docker-in-Docker	Test execution sandbox
Source Control	GitHub / GitLab / Azure Repos	Test file versioning
CI/CD Output	JUnit XML	Universal CI result format

Frontend

Component	Technology	Role
UI Framework	React + TypeScript	Dashboard
Real-time	WebSocket	Execution status updates
State Management	Zustand or React Query	Client-side state

20. Repository Structure

```
aegisqa/
│
├── backend/
│   ├── api/                        # FastAPI routes and middleware
│   │   ├── routes/                 # tickets, execute, results, memory, reports
│   │   ├── auth/                   # JWT, RBAC
│   │   └── websockets/             # real-time execution status
│   │
│   ├── graph/                      # LangGraph definitions
│   │   ├── state.py                # TestContext Pydantic model
│   │   ├── nodes/                  # one file per node (load_ticket.py, etc.)
│   │   ├── routing.py              # conditional edge functions
│   │   └── workflow.py             # graph assembly
│   │
│   ├── agents/                     # Agent class definitions
│   │   ├── base.py                 # BaseAgent with registry decorator
│   │   ├── requirement_agent.py
│   │   ├── coverage_planner.py
│   │   ├── test_case_generator.py
│   │   ├── test_data_resolver.py
│   │   ├── automation_generator.py
│   │   ├── validation_agent.py
```

```
│   │   ├── investigation_coordinator.py
│   │   ├── locator_repair_agent.py
│   │   ├── report_generator.py
│   │   └── memory_agent.py
│   │
│   ├── skills/                     # Skill class definitions
│   │   ├── base.py
│   │   ├── requirement/
│   │   ├── coverage/
│   │   ├── test_generation/
│   │   ├── test_data/
│   │   ├── automation/
│   │   ├── validation/
│   │   ├── investigation/
│   │   ├── reporting/
│   │   └── memory/
│   │
│   ├── tools/                      # Tool class definitions
│   │   ├── base.py
│   │   ├── jira_tool.py
│   │   ├── git_tool.py
│   │   ├── robot_tool.py
│   │   ├── browser_tool.py
│   │   ├── api_tool.py
│   │   ├── database_tool.py
│   │   ├── filesystem_tool.py
│   │   ├── vault_tool.py
│   │   ├── vectordb_tool.py
│   │   ├── llm_tool.py
│   │   └── knowledge_search_tool.py
│   │
│   ├── memory/                     # Memory layer implementations
│   │   ├── vector_store.py         # VectorStore protocol + PgVectorStore
│   │   ├── episodic.py
│   │   ├── semantic.py
│   │   ├── skill_memory.py
│   │   └── freshness.py            # Freshness score calculation
│   │
│   ├── rag/                        # RAG pipeline
│   │   ├── indexer.py              # Document ingestion and embedding
│   │   ├── retriever.py            # Query → embed → search → rerank
│   │   ├── reranker.py             # ms-marco-MiniLM wrapper
│   │   └── indexes/                # Index configuration per source
│   │
│   ├── workers/                    # Celery worker definitions
│   │   ├── celery_app.py
│   │   ├── agent_worker.py
│   │   ├── execution_worker.py
│   │   └── memory_worker.py
│   │
│   ├── execution/                  # Robot Framework execution wrappers
│   │   ├── runner.py
│   │   ├── collector.py
│   │   └── artifact_store.py
│   │
```

```
│   ├── prompts/                    # Jinja2 prompt templates
│   │   ├── requirement_analysis.j2
│   │   ├── test_case_generation.j2
│   │   ├── automation_generation.j2
│   │   ├── investigation.j2
│   │   └── reporting.j2
│   │
│   ├── config/                     # Configuration files
│   │   ├── models.yaml             # Model tier assignments
│   │   ├── environments.yaml       # Staging/prod environment configs
│   │   └── settings.py             # Pydantic settings model
│   │
│   └── plugins/                    # Optional plugin directory
│       ├── playwright_plugin/
│       ├── appium_plugin/
│       └── nemo_guardrails/
│
├── frontend/
│   ├── src/
│   │   ├── pages/                  # Dashboard, Tickets, Execution, Memory
│   │   ├── components/             # ReviewPanel, ExecutionStatus, ReportView
│   │   └── hooks/                  # useWebSocket, useExecution
│   └── public/
│
├── infrastructure/
│   ├── docker-compose.yml          # All services
│   ├── docker-compose.prod.yml
│   └── vault/                      # Vault configuration
│
├── tests/                          # Platform unit + integration tests
│   ├── unit/
│   │   ├── agents/
│   │   ├── skills/
│   │   └── tools/
│   └── integration/
│
├── docs/
│   ├── architecture.md
│   ├── agent-guide.md
│   ├── skill-guide.md
│   └── deployment.md
│
└── README.md
```

---

21. CI/CD Integration

Webhook API

```
POST /api/v1/execute
Authorization: Bearer {token}
Content-Type: application/json

{
  "suite": "regression",          // "smoke" | "regression" | "full" | ticket ID
  "branch": "feature/JIRA-123",  // optional: run tests from specific branch
  "env": "staging",               // environment alias from environments.yaml
  "tags": ["JIRA-123", "payment"] // optional: Robot Framework tag filters
}

Response 202 Accepted:
{
  "run_id": "exec-uuid-001",
  "status_url": "/api/v1/results/exec-uuid-001",
  "websocket_url": "ws://aegisqa.internal/ws/exec-uuid-001"
}
```

Results API

```
GET /api/v1/results/{run_id}
GET /api/v1/results/{run_id}/junit.xml
GET /api/v1/results/{run_id}/report.html
GET /api/v1/results/{run_id}/summary.json
```

GitHub Actions

```
- name: Trigger AegisQA and publish results
  run: |
    RUN_ID=$(curl -sf -X POST $AEGISQA_URL/api/v1/execute \
      -H "Authorization: Bearer $AEGISQA_TOKEN" \
      -d '{"suite":"smoke","env":"staging"}' | jq -r .run_id)

    # Poll until complete (max 10 minutes)
    for i in $(seq 1 60); do
      STATUS=$(curl -sf $AEGISQA_URL/api/v1/results/$RUN_ID | jq -r .status)
      [ "$STATUS" = "completed" ] || [ "$STATUS" = "completed_with_failures" ] && break
      sleep 10
    done

    curl -sf $AEGISQA_URL/api/v1/results/$RUN_ID/junit.xml > test-results.xml
```

GitLab CI

```
aegisqa_smoke:
  stage: test
  script:
    - |
      RUN_ID=$(curl -sf -X POST $AEGISQA_URL/api/v1/execute \
        -H "Authorization: Bearer $AEGISQA_TOKEN" \
        -d '{"suite":"smoke","env":"staging"}' | jq -r .run_id)
      curl -sf $AEGISQA_URL/api/v1/results/$RUN_ID/junit.xml > junit.xml
  artifacts:
    reports:
      junit: junit.xml
```

Azure DevOps

```
- task: Bash@3
  displayName: Run AegisQA
  inputs:
    script: |
      RUN_ID=$(curl -sf -X POST $(AEGISQA_URL)/api/v1/execute \
        -H "Authorization: Bearer $(AEGISQA_TOKEN)" \
        -d '{"suite":"regression","env":"staging"}' | jq -r .run_id)
      curl -sf $(AEGISQA_URL)/api/v1/results/$RUN_ID/junit.xml > $(Build.ArtifactStagingDirectory)/junit.xml

- task: PublishTestResults@2
  inputs:
    testResultsFormat: JUnit
    testResultsFiles: '$(Build.ArtifactStagingDirectory)/junit.xml'
```

---

22. MVP Scope — Phase 1

Build only what delivers the core pipeline end-to-end. Avoid the temptation to build all 13 modules.

Included in MVP:

Module	Rationale
Dashboard & Ticket Management	Entry point; required for everything else
Requirement Analysis Agent	Core value — ticket → structured requirements
Coverage Planner	Improves test quality; relatively simple
Test Case Generation Agent	Core value
Test Data Management (factory + fixture only)	Required for tests to actually run
Automation Generation Agent + Validator	Core value; dry-run validation prevents review waste
Human Validation Layer + Git PR	Safety gate + Git-native output
Execution Engine (dashboard trigger + one CI webhook)	Core value — make tests run
Investigation Agent (log + API evidence, no network)	Core value — make failures intelligible
AI Reporting Agent	Stakeholder-facing value
Memory Agent (episodic store only)	Foundation for Phase 2 improvements

Excluded from MVP:

Module	Phase
Self-Healing / Locator Repair Agent	Phase 2
Full RAG pipeline (Confluence, Git, API specs)	Phase 2
Semantic and skill memory layers	Phase 2
Wireshark / tshark Network Analysis	Phase 3
Mobile testing (Appium)	Phase 4
NeMo Guardrails	Phase 3 (optional)
Advanced Celery worker scaling	Phase 3

Business value delivered by MVP:
The complete pipeline from ticket to failure report is functional. Engineers review and approve AI-generated tests. Tests run in CI. Failures produce evidence-based reports. The system starts accumulating episodic memory immediately, so Phase 2 improvements land on a populated knowledge base.

---

23. Post-MVP Roadmap

Phase 2 — Intelligence Layer

- Full Hermes memory architecture (semantic + skill memory layers)
- Complete RAG pipeline: Confluence indexing, Git history indexing, API spec indexing
- Self-Healing / Locator Repair Agent
- Similar failure retrieval using episodic memory (improves investigation quality)
- Memory management dashboard (view, flag, invalidate entries)
- Horizontal Celery worker scaling documentation and configuration

Phase 3 — Advanced Investigation

- Wireshark / tshark Network Analysis Agent
- HAR file correlation with test failures
- Database state snapshot comparison (before/after per test)
- NeMo Guardrails integration (optional safety layer for agent outputs)
- Performance test integration (k6 or Locust keyword generation)
- Advanced multi-agent parallelism (run multiple tickets concurrently)

Phase 4 — Extended Platform

- Mobile testing: Appium keyword generation and execution
- STF (Smartphone Test Farm) integration
- Citrix / virtual desktop application testing
- Plugin marketplace: community-contributed tools and skills
- Multi-organization support (SaaS deployment)

Phase 5 — Enterprise

- Fine-tuned domain-specific models (optional: train on organization's own tickets and scripts)
- SAML / SSO integration
- Advanced RBAC with project-level permissions
- Compliance reporting (test coverage by regulatory requirement)
- SLA dashboards for executive reporting cadences

---

24. NeMo Guardrails — Optional Safety Layer

NVIDIA NeMo Guardrails is a framework for constraining LLM output to defined behavioral boundaries. In AegisQA, it is an optional Phase 3 addition that sits between the LLMTool and any agent that can produce consequential output.

What it prevents:

Risk	Guardrail
Agent generating a script that modifies production data	Block any `DELETE`, `DROP`, or production connection strings in generated Robot files
Agent bypassing human approval via a generated script	Detect and block scripts with `Set Global Variable` patterns that override approval flags
Agent generating commands that access out-of-scope resources	Restrict generated scripts to the environments defined in `environments.yaml`
Prompt injection via ticket content	Filter ticket text before injecting into agent prompts

Implementation:

```
# backend/plugins/nemo_guardrails/guardrail_llm_tool.py
from nemoguardrails import RailsConfig, LLMRails

class GuardrailedLLMTool(LLMTool):
    def __init__(self):
        config = RailsConfig.from_path("plugins/nemo_guardrails/rails/")
        self.rails = LLMRails(config)

    async def complete(self, prompt: str, model_tier: str) -> str:
        return await self.rails.generate_async(prompt=prompt)
```

This is a plugin — it replaces `LLMTool` in any agent where it is configured, without changing agent or skill code.

---

25. Architectural Decisions Log

Decisions made explicitly in this document so they are not relitigated later:

Decision	Choice	Rationale	Revisit Trigger
Orchestration engine	LangGraph	Explicit state graph, inspectable, typed shared state	If multi-organization parallelism requirements emerge
Agent framework	Custom (registry pattern, not CrewAI/AutoGen)	Full control over state, routing, isolation	If team grows rapidly and needs higher-level abstraction
Vector store (Phase 1)	pgvector	Same DB, simpler ops, adequate for < 500K vectors	If total vectors > 500K or p95 query latency > 200ms
Vector store migration target	Qdrant	Best-in-class purpose-built vector DB	—
Re-ranker	ms-marco-MiniLM-L-6-v2	Runs locally, strong precision, < 100ms latency	If throughput becomes bottleneck
Model for code generation	GPT-4 class only	Local models (7B-13B) fail reliably on Robot Framework	If Llama 3 70B+ local hosting becomes feasible
Confidence scoring	Deterministic evidence weights	Explainable, non-hallucinated, auditable	— (principle, not technology)
Event Bus	Redis Pub/Sub	Already in stack for Celery; fire-and-forget notifications	If event persistence becomes required (→ Kafka)
Celery role	Long-running task execution	Separate concern from notifications	—
Secrets	HashiCorp Vault	Industry standard; no plaintext credentials anywhere	Cloud-only: AWS Secrets Manager or GCP Secret Manager
Git integration	Branch + PR per approval	Tests in version control from day one; standard review UX	—
Memory staleness	Exponential decay + TTL flagging	No auto-delete; engineers validate before expiry	—
NeMo Guardrails	Optional plugin, Phase 3	Not required for MVP; adds latency; valuable for enterprise	If enterprise compliance requirements emerge

26. Version Changelog

Version	Summary
v1	Initial concept: 10 modules, basic pipeline, Robot Framework generation, LangGraph/CrewAI/AutoGen listed without decision
v2	Fixed: confidence scoring → deterministic weights; added test data management module; honest model tier assignment; CI/CD integration; TestContext schema; Git PR integration; dry-run validation; Wireshark moved to post-MVP
v3	Added: LangGraph explicit graph with 12 nodes; OpenClaw-inspired runtime (Gateway, registries, Event Bus, workers); Hermes-inspired 5-layer memory; RAG pipeline with re-ranker and vector DB decision; Coverage Planner as dedicated agent; memory freshness model; tool isolation per tool type; Event Bus vs Celery distinction; memory_refs schema; NeMo Guardrails consideration; full agent, skill, and tool catalogs